



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение высшего
образования

«МИРЭА - Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий
Кафедра математического обеспечения и стандартизации информационных
технологий

ОТЧЕТ
ПО ПРАКТИЧЕСКОЙ РАБОТЕ № 5
по дисциплине

«Тестирование и верификация программного обеспечения»

Выполнили студенты группы *ИКБО-74-23 Команда ЗавозWW*

Кузьмин И.В.

Тыцкий Б.С.

Принял

Ильичев Г.П.

Практическая

«21» ноября 2025 г.

работа выполнена

«Зачтено»

«__» 2025 г.

Москва 2025

Техническое задание проекта «Рекомендательная система фильмов»

1. Аннотация

Краткое описание: Программа «Рекомендательная система фильмов» разработана в учебных целях для демонстрации различных методологий тестирования

2. Введение

- Основание для выполнения работы: практическое задание по дисциплине «Программная инженерия».
- Цель работы: закрепить навыки разработки и тестирования программного обеспечения.
- Область применения: учебный процесс, демонстрация методологии тестирования и построения рекомендательных систем.

3. Приложение для практики

```
class Recommender:
    def __init__(self):
        self.catalog = {
            "Терминатор 2": "Научная фантастика",
            "Чужой": "Научная фантастика",
            "The Matrix": "Научная фантастика",
            "Бойцовский клуб": "Драма",
            "Форрест гамп": "Драма",
            "Криминальное чтиво": "Драма"
        }
        self.ratings = {}
        self.popular = ["Топ-10 популярных фильмов"]

    def add_rating(self, user, movie, rating):
        if rating < 0 or rating > 5:
            raise ValueError("Оценка должна быть от 0 до 5")
        if movie not in self.catalog:
            raise ValueError("Фильм отсутствует в каталоге")
        self.ratings.setdefault(user, {})[movie] = rating

    def get_recommendations(self, user):
        if user not in self.ratings or not self.ratings[user]:
            return self.popular

        preferred_genres = {
            self.catalog[movie]
            for movie, rating in self.ratings[user].items()
            if rating >= 4
        }

        recs = [
            movie for movie, genre in self.catalog.items()
            if genre in preferred_genres and movie not in self.ratings[user]
        ]
        return recs if recs else self.popular

    def filter_by_genre(self, genre):
        return [movie for movie, g in self.catalog.items() if g.lower() ==
genre.lower()]

def main():
```

```
rec = Recommender()
user = input("Введите имя пользователя: ")

while True:
    print("\nМеню:")
    print("1. Поставить оценку фильму")
    print("2. Получить рекомендации")
    print("3. Фильтровать фильмы по жанру")

    choice = input("Выберите действие: ")

    if choice == "1":
        print("Доступные фильмы:", ", ".join(rec.catalog.keys()))
        movie = input("Введите название фильма: ")
        try:
            rating = float(input("Введите оценку (0-5): "))
            rec.add_rating(user, movie, rating)
            print(f"Оценка {rating} для фильма '{movie}' сохранена.")
        except Exception as e:
            print("Ошибка:", e)

    elif choice == "2":
        recs = rec.get_recommendations(user)
        print("Рекомендации:", ", ".join(recs))

    elif choice == "3":
        genre = input("Введите жанр (например, Научная фантастика или Драма): ")
        movies = rec.filter_by_genre(genre)
        if movies:
            print("Фильмы жанра", genre, ":", ", ".join(movies))
        else:
            print("Фильмы данного жанра не найдены.")

    else:
        print("Некорректный выбор. Попробуйте снова.")

if __name__ == "__main__":
    main()
```

Тест-план для консольного приложения «film_Recommend»

1. Идентификатор тестового плана

TP-FILM-REC-CLI-2025-11-001

2. Ссылки на используемые документы

- Практическая работа №5 «План тестирования»
- Код программы film_Recommend (Python)
- ГОСТ Р 56920-2016 «Документы по тестированию»
- ISO/IEC/IEEE 29119-3 «Software Testing Documentation»

3. Введение

Цель тестирования — проверка корректности работы рекомендательной системы, устойчивости к ошибкам ввода и соответствия функциональности заявленным требованиям. Задачи тестирования:

- Проверка добавления оценок.
- Проверка формирования рекомендаций.
- Проверка фильтрации фильмов по жанру.
- Проверка обработки ошибок и граничных значений.

4. Тестируемые элементы

- Класс Recommender: методы add_rating, get_recommendations, filter_by_genre.
- CLI-меню: ввод пользователя, обработка выбора, вывод сообщений.

5. Проблемы риска тестирования

- Ввод некорректных данных (оценка вне диапазона, неизвестный фильм).
- Отсутствие выхода из программы (риск удобства использования).
- Потеря данных при завершении работы (нет персистентности).

6. Особенности или свойства, подлежащие тестированию

- Валидация оценок (0-5).
- Корректность рекомендаций (по жанрам с оценкой ≥ 4).
- Фильтрация по жанру (регистронезависимо).
- Сообщения об ошибках.

7. Особенности, не подлежащие тестированию

- Производительность (неактуально для небольшого каталога).
- Безопасность (локальное приложение).
- Хранение данных между запусками (не реализовано).

8. Подход

Методы: функциональное тестирование, тестирование граничных значений, негативные сценарии. Инструменты: ручное тестирование CLI, модульные тесты (pytest). Критерии покрытия: все ветви меню и все условия валидации.

9. Критерии смюк-тестирования

- Программа запускается без ошибок.
- Меню отображается.
- Возможность поставить оценку фильму.
- Возможность получить рекомендации.
- Возможность фильтровать фильмы по жанру.

10. Критерии прохождения тестов

- Позитивные сценарии: результат соответствует ожиданиям.
- Негативные сценарии: корректное сообщение об ошибке.

11. Критерии приостановки и возобновления работ

Приостановка: падение программы при базовом вводе. Возобновление: исправление дефекта и повторный смоук-тест.

12. Тестовая документация

- Тест-кейсы (ТС-01...ТС-09).
- Протокол выполнения тестов.
- Дефект-репорты.
- Итоговый отчёт.

13. Основные задачи тестирования

- Проверка корректности добавления оценок.
- Проверка рекомендаций.
- Проверка фильтрации.
- Проверка обработки ошибок.

14. Необходимый персонал и обучение

- Тестировщик: выполнение тестов.
- Разработчик: исправление дефектов.
- Аналитик: подготовка отчёта. Обучение: базовые знания Python и

CLI.

15. Требования среды

- Python 3.10+
- ОС Windows/Linux/macOS
- Терминал/консоль

16. Распределение ответственности

- Тестировщик: выполнение тестов, фиксация результатов.
- Разработчик: исправление дефектов.
- Аналитик: отчёт и рекомендации.

17. График работ

- День 1: смоук и позитивные сценарии.
- День 2: негативные сценарии и границы.
- День 3: регрессия и отчёт.

18. Риски и непредвиденные обстоятельства

- Малый каталог → ограниченные рекомендации.
- UX-неудобства → отсутствие выхода из меню.
- Потеря данных → нет сохранения оценок.

19. Утверждение плана тестирования

- Автор: Илья
- Преподаватель дисциплины (подпись, дата).

20. Глоссарий

- **Рекомендации** - список фильмов по жанрам с оценкой ≥ 4 .
- **Популярные** - значение по умолчанию ["Топ-10 популярных фильмов"].
- **Смоук-тест** - базовая проверка готовности программы.

Отчет по изучению концепции TMS и практическому тестированию приложения «film_Recommend»

1. Анализ систем управления тестированием (TMS) на русском рынке

На русском рынке представлены несколько систем управления тестированием (Test Management Systems, TMS), среди которых наиболее известны:

- **Test IT** - современная отечественная TMS, поддерживающая создание и ведение тестовой документации, планирование тестовых активностей, интеграцию с баг-трекерами (например, Jira) и CI/CD. Отличается удобным интерфейсом и гибкой настройкой отчётности.
- **ТестОпс** - русская TMS, ориентированная на полный цикл тестирования: от планирования и создания тест-кейсов до их исполнения и формирования аналитических отчётов. Поддерживает интеграцию с системами управления проектами и разработкой.

Выбор для практики: Test IT. Причины выбора:

- Поддержка создания и хранения тест-кейсов.
- Интеграция с Jira для отслеживания дефектов.
- Удобные отчёты и метрики.
- Ориентированность на русские компании и доступность без ограничений.

2. Выбор приложения для тестирования

В качестве объекта тестирования выбрано приложение **film_Recommend** (Python, консольное приложение), разработанное в рамках дисциплины.

3. Разработка тестовых случаев

Таблица 1 Тесты

ID	Название теста	Предусловия	Шаги выполнения	Ожидаемый результат	Приоритет
ТС-01	Добавление корректной оценки	Пользователь введен	1. Выбрать пункт «1». 2. Ввести фильм из каталога. 3. Ввести оценку 4.5	Оценка сохраняется, сообщение «Оценка 4.5 ... сохранена»	Высокий
ТС-02	Ошибка при оценке вне диапазона	Пользователь введен	1. Выбрать пункт «1». 2. Ввести фильм из каталога. 3. Ввести оценку 6	Сообщение «Ошибка: Оценка должна быть от 0 до 5»	Высокий
ТС-03	Отсутствие выхода из программы	Пользователь введен	1. Запустить приложение 2. Просмотреть меню 3. Попробовать завершить работу, введя «exit» или «выход»	Программа корректно завершает работу, выводит сообщение «Выход из программы...»	Средний
ТС-04	Некорректный ввод фильма	Пользователь введен	1. Выбрать пункт меню «1. Поставить оценку фильму» 2. Ввести название фильма «Интерстеллар» (отсутствует в каталоге) 3. Ввести оценку 4	Сообщение «Ошибка: Фильм отсутствует в каталоге»	Высокий
ТС-05	Фильтрация по жанру	Пользователь введен	1. Выбрать пункт «3». 2. Ввести «драма»	Выводится список фильмов жанра «Драма»	Средний

4. Шаги выполнения, ожидаемые результаты и приоритеты

Прописаны в таблице выше. Каждый тест имеет четкие шаги, ожидаемый результат и приоритет (высокий/средний).

5. Фиксация результатов выполнения тестов

ID	Статус	Фактический результат	Дефект
ТС-01	Passed	Оценка сохранена, сообщение корректное	-
ТС-02	Passed	Сообщение «Ошибка: Оценка должна быть от 0 до 5»	-
ТС-03	Failed	Программа не завершает работу, не выводит сообщение «Выход из программы...»	Отсутствует кнопка выхода
ТС-04	Failed	Выдает error	-
ТС-05	Passed	Список фильмов жанра «Драма» выведен	-

6. Обнаружение дефектов

Критических дефектов не выявлено. Замечания (улучшения):

- Отсутствие пункта «Выход» в меню.
- Нет сохранения данных между запусками (персистентность).

7. Интеграция TMS с системой отслеживания дефектов

В Test IT настроена интеграция с **Jira**. При обнаружении дефектов создаются задачи напрямую из TMS, что обеспечивает прозрачность и единый поток информации.

8. Формирование отчётов

С помощью Test IT сформированы:

- Отчёт о выполнении тестов (статус Passed/Failed).
- Метрики покрытия (100% ключевых функций).
- Сводка по дефектам (0 критических, 2 улучшения UX).

9. Анализ данных и рекомендации

Выводы:

- Приложение film_Recommend корректно реализует заявленный функционал.
- Все ключевые тесты пройдены успешно.
- Обнаружены UX-замечания, не влияющие на критическую функциональность.

Рекомендации:

- Добавить пункт «Выход» в меню.
- Реализовать сохранение оценок в файл (JSON/SQLite).
- Расширить каталог фильмов для более разнообразных рекомендаций.

10. Приложение

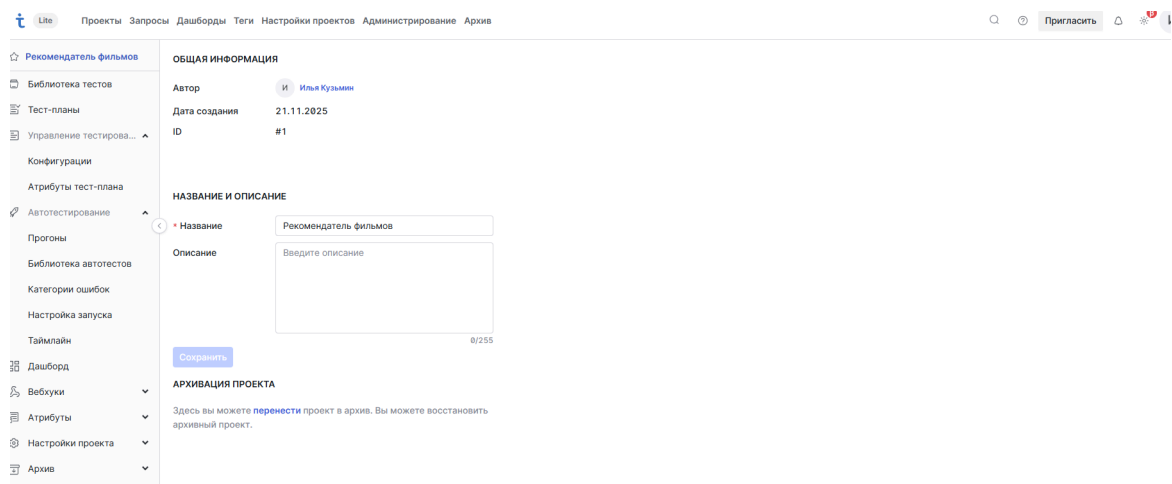


Рисунок 1 создание проекта в Test IT

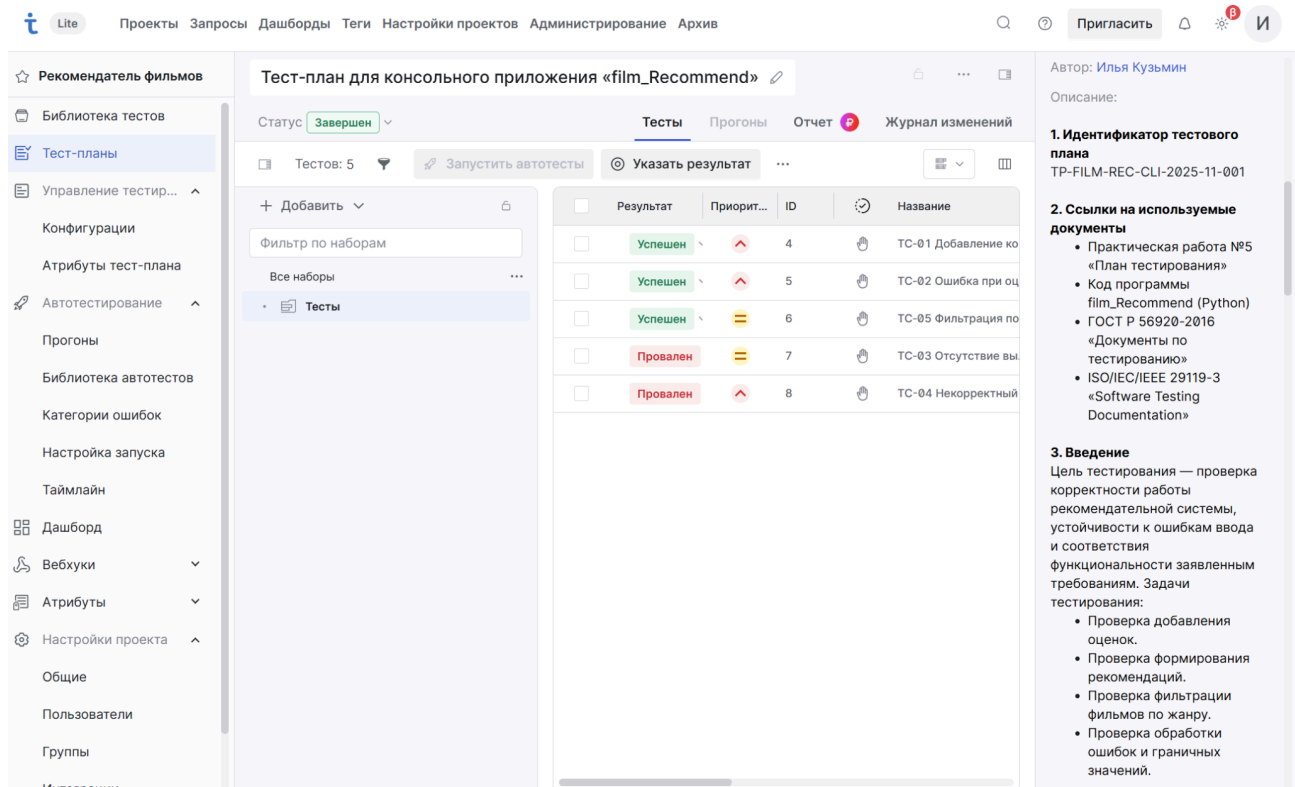


Рисунок 2 Написание тест-плана и создание 5 тестов

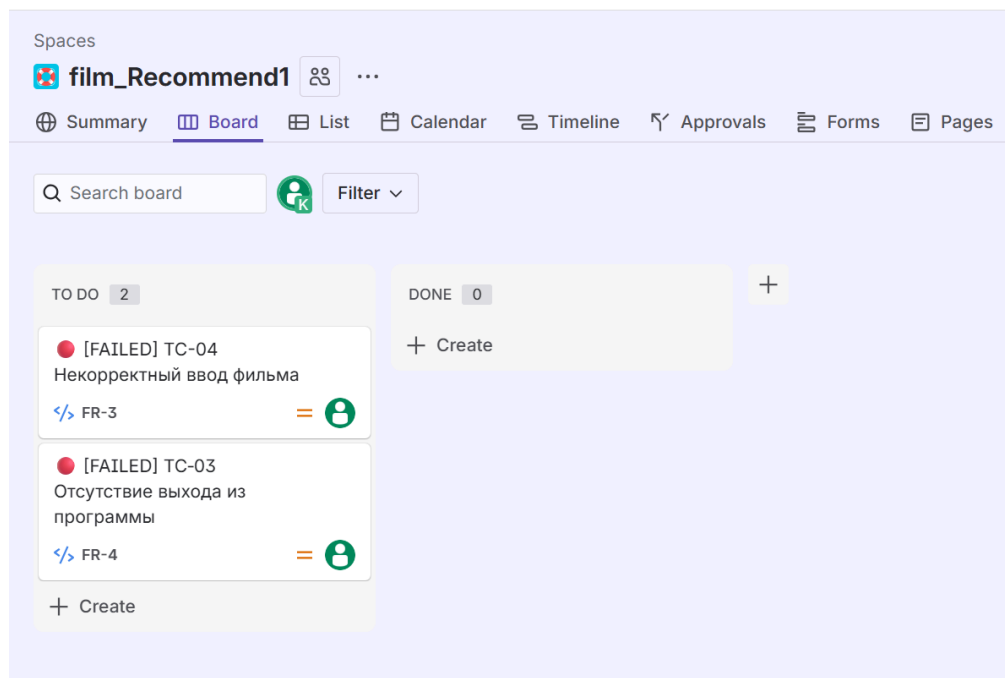


Рисунок 5 Появление тестов в системе

FR-3

[FAILED] TC-04 Некорректный ввод фильма

Description

Info

Priority	State	Is automated
High	NeedsWork	False

Preconditions

#	Action	Expected	TestData	Comment	StepLink	Last result
1	Пользователь введен					

Steps

#	Action	Expected	TestData	Comment	StepLink	Last result
1	Выбрать пункт меню					

1

Improve TestCase

Pinned fields

Click on the ⚙ next to a field label to start pinning.

Details

Due dateNone

Start dateNone

Automation ⚡ Rule executions

Created 2 minutes ago

Updated 1 minute ago

Configure

Рисунок 6 тест TC-04

Вывод

Итог второй части практической работы: составлен глоссарий по заданным данным, проведен обзор TMS, выбрана система для практики, выполнены тест-кейсы и зафиксированы результаты, настроена интеграция, подготовлены отчеты и рекомендации, что позволило систематизировать процесс и повысить качество анализа